3FS

Deep Dive into DeepSeek — Chapter 30

Yin Yang

Foundation Books Project

The one rule of this chapter

A checkpoint write is not a checkpoint until a coherent generation can be identified, read, and restored.

- 3FS is a shared file interface plus stateless metadata services, built for SSD and RDMA paths, combining chunk placement with CRAQ-style replication.
- "The last byte landed" and "this generation is safe to restore" are different claims — the whole chapter is about the gap between them.
- The running example: a 10^{12}-parameter checkpoint, 1,024 writers, an hourly write interval, a ten-minute restore target.

The one rule of this chapter

A checkpoint write is not a checkpoint until a coherent generation can be identified, read, and restored.

- 3FS is a shared file interface plus stateless metadata services, built for SSD and RDMA paths, combining chunk placement with CRAQ-style replication.
- "The last byte landed" and "this generation is safe to restore" are different claims — the whole chapter is about the gap between them.
- The running example: a 10^{12}-parameter checkpoint, 1,024 writers, an hourly write interval, a ten-minute restore target.

Goal: follow one checkpoint from namespace metadata through chunk placement, chain replication, client I/O, and target-local recovery to a restore-safe published generation.

Roadmap

Storage on the accelerator critical path

System boundary and namespace

Consistency and checkpoint publication

Target-local recovery and client I/O

Workloads, benchmarks, and operations

Storage on the accelerator critical path

3FS inside Fire-Flyer 2

Fire-Flyer 2 layer	Paper-reported role
Compute and training	About 1,250 nodes, 8 PCIe A100 GPUs each; HFReduce and HaiScale run collective communication and training.
Shared fabric	Two-zone InfiniBand fat-tree; service levels and static routing separate storage traffic from collective traffic.
3FS storage	180 dual-homed storage nodes, 16 NVMe SSDs and two 200-Gbps NICs each.

3FS inside Fire-Flyer 2

Fire-Flyer 2 layer	Paper-reported role
Compute and training	About 1,250 nodes, 8 PCIe A100 GPUs each; HFReduce and HaiScale run collective communication and training.
Shared fabric	Two-zone InfiniBand fat-tree; service levels and static routing separate storage traffic from collective traffic.
3FS storage	180 dual-homed storage nodes, 16 NVMe SSDs and two 200-Gbps NICs each.
HAI Platform	Schedules jobs; its checkpoint manager batches 3FS reads and writes after GPU-to-host staging.

3FS inside Fire-Flyer 2

Fire-Flyer 2 layer	Paper-reported role
Compute and training	About 1,250 nodes, 8 PCIe A100 GPUs each; HFReduce and HaiScale run collective communication and training.
Shared fabric	Two-zone InfiniBand fat-tree; service levels and static routing separate storage traffic from collective traffic.
3FS storage	180 dual-homed storage nodes, 16 NVMe SSDs and two 200-Gbps NICs each.
HAI Platform	Schedules jobs; its checkpoint manager batches 3FS reads and writes after GPU-to-host staging.
3FS-KV	A separately named data layer above 3FS: key-value, message-queue, and object-storage models.

3FS inside Fire-Flyer 2

Fire-Flyer 2 layer	Paper-reported role
Compute and training	About 1,250 nodes, 8 PCIe A100 GPUs each; HFReduce and HaiScale run collective communication and training.
Shared fabric	Two-zone InfiniBand fat-tree; service levels and static routing separate storage traffic from collective traffic.
3FS storage	180 dual-homed storage nodes, 16 NVMe SSDs and two 200-Gbps NICs each.
HAI Platform	Schedules jobs; its checkpoint manager batches 3FS reads and writes after GPU-to-host staging.
3FS-KV	A separately named data layer above 3FS: key-value, message-queue, and object-storage models.

Every 3FS read competes with HFReduce and NCCL on the same fabric — storage supply cannot be read off SSD bandwidth alone. [REPORTED: Fire-Flyer AI-HPC paper, arXiv:2408.14158]

Demand, supply, and the toy checkpoint

$$\text{storage demand} = \text{workload bytes} + \text{metadata ops} + \text{recovery slack}$$

$$\text{storage supply} = \min(\beta_{\text{ssd}}, \beta_{\text{net}}, \beta_{\text{service}}, \beta_{\text{metadata}}, \beta_{\text{client}})$$

Demand, supply, and the toy checkpoint

$$\text{storage demand} = \text{workload bytes} + \text{metadata ops} + \text{recovery slack}$$

$$\text{storage supply} = \min(\beta_{\text{ssd}}, \beta_{\text{net}}, \beta_{\text{service}}, \beta_{\text{metadata}}, \beta_{\text{client}})$$

- The running example: 10^{12} parameters, 2 stored weight bytes and 8 optimizer-state bytes per parameter, 0.5 TiB extra payload, 1,024 writers, an hourly interval, a ten-minute restore target, a 1.5 safety factor.

Demand, supply, and the toy checkpoint

$$\text{storage demand} = \text{workload bytes} + \text{metadata ops} + \text{recovery slack}$$

$$\text{storage supply} = \min(\beta_{\text{ssd}}, \beta_{\text{net}}, \beta_{\text{service}}, \beta_{\text{metadata}}, \beta_{\text{client}})$$

- The running example: 10^{12} parameters, 2 stored weight bytes and 8 optimizer-state bytes per parameter, 0.5 TiB extra payload, 1,024 writers, an hourly interval, a ten-minute restore target, a 1.5 safety factor.
- Supply is the *slowest* limiting term — a large reported read number never overrides a saturated metadata path.

Demand, supply, and the toy checkpoint

$$\text{storage demand} = \text{workload bytes} + \text{metadata ops} + \text{recovery slack}$$

$$\text{storage supply} = \min(\beta_{\text{ssd}}, \beta_{\text{net}}, \beta_{\text{service}}, \beta_{\text{metadata}}, \beta_{\text{client}})$$

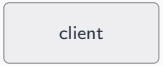
- The running example: 10^{12} parameters, 2 stored weight bytes and 8 optimizer-state bytes per parameter, 0.5 TiB extra payload, 1,024 writers, an hourly interval, a ten-minute restore target, a 1.5 safety factor.
- Supply is the *slowest* limiting term — a large reported read number never overrides a saturated metadata path.

A storage claim needs the demand terms, the bottlenecked supply path, and the operating objective in the same record.

System boundary and namespace

The 3FS system boundary

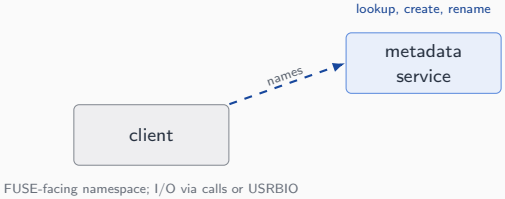
Definition (3FS system boundary)
Joins a client-visible file operation and the distributed state needed to complete it: namespace metadata, chunk layout, chain placement, replica state, completion state, and operational metrics.



FUSE-facing namespace; I/O via calls or USRBIO

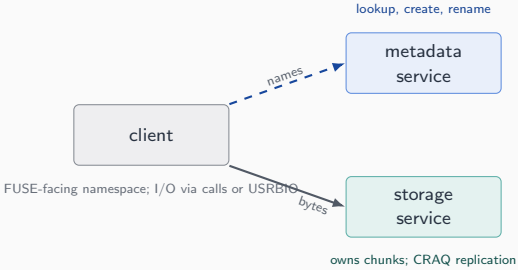
The 3FS system boundary

Definition (3FS system boundary)
Joins a client-visible file operation and the distributed state needed to complete it: namespace metadata, chunk layout, chain placement, replica state, completion state, and operational metrics.



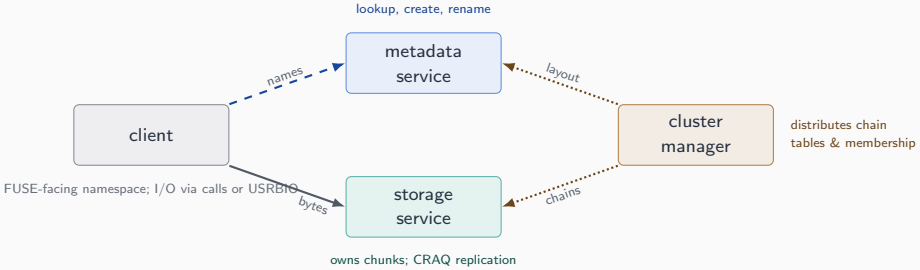
The 3FS system boundary

Definition (3FS system boundary)
Joins a client-visible file operation and the distributed state needed to complete it: namespace metadata, chunk layout, chain placement, replica state, completion state, and operational metrics.



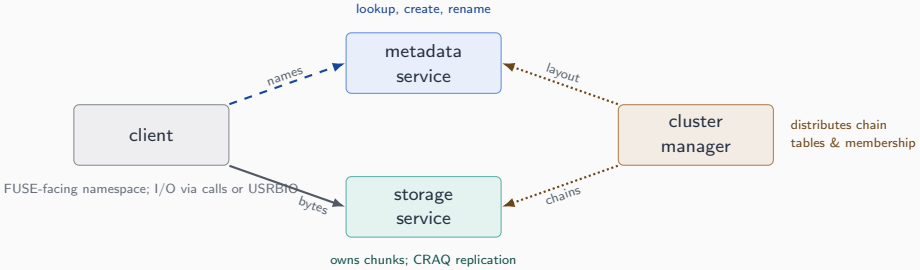
The 3FS system boundary

Definition (3FS system boundary)
Joins a client-visible file operation and the distributed state needed to complete it: namespace metadata, chunk layout, chain placement, replica state, completion state, and operational metrics.



The 3FS system boundary

Definition (3FS system boundary)
Joins a client-visible file operation and the distributed state needed to complete it: namespace metadata, chunk layout, chain placement, replica state, completion state, and operational metrics.



HFReduce, HaiScale, and the HAI Platform sit outside this boundary: they create traffic and decide completion, but 3FS owns the namespace, chunk state, and replica protocol.

Chunks, chains, and three version domains

Version type	Scope
`ChainTableVersion`	Entire chain-table snapshot; a file layout retains the table version it was read under.

Chunks, chains, and three version domains

Version type	Scope
`ChainTableVersion`	Entire chain-table snapshot; a file layout retains the table version it was read under.
`ChainVersion`	Per-chain membership epoch; a manager update advances it.

Chunks, chains, and three version domains

Version type	Scope
`ChainTableVersion`	Entire chain-table snapshot; a file layout retains the table version it was read under.
`ChainVersion`	Per-chain membership epoch; a manager update advances it.
`ChunkVer`	Per-chunk content generation; advances on data updates.

Chunks, chains, and three version domains

Version type	Scope
`ChainTableVersion`	Entire chain-table snapshot; a file layout retains the table version it was read under.
`ChainVersion`	Per-chain membership epoch; a manager update advances it.
`ChunkVer`	Per-chunk content generation; advances on data updates.

- A chunk is $\lceil B_{\text{worker}}/S_{\text{chunk}} \rceil$ pieces of one shard on an ordered chain; placement spreads a failed disk's redirected reads across many survivors, not just two.
- Metadata pressure is a separate axis from bytes: 8,192 shards need at least 16,385 metadata operations before any byte moves.

Chunks, chains, and three version domains

Version type	Scope
`ChainTableVersion`	Entire chain-table snapshot; a file layout retains the table version it was read under.
`ChainVersion`	Per-chain membership epoch; a manager update advances it.
`ChunkVer`	Per-chunk content generation; advances on data updates.

- A chunk is $\lceil B_{\text{worker}}/S_{\text{chunk}} \rceil$ pieces of one shard on an ordered chain; placement spreads a failed disk's redirected reads across many survivors, not just two.
- Metadata pressure is a separate axis from bytes: 8,192 shards need at least 16,385 metadata operations before any byte moves.

These three version counters need not advance together — a chunk-content retry cannot repair a stale table or a stale chain epoch.

Consistency and checkpoint publication

CRAQ-style write-all, read-any

Definition (CRAQ-style replication)
Writes propagate through every replica in an ordered chain; reads may use any replica whose state is safe for the requested semantics.

$$C_v \xrightarrow{\text{prepare}} (C_v, U_{v+1})$$

CRAQ-style write-all, read-any

Definition (CRAQ-style replication)
Writes propagate through every replica in an ordered chain; reads may use any replica whose state is safe for the requested semantics.

$$C_v \xrightarrow{\text{prepare}} (C_v, U_{v+1}) \xrightarrow{\text{tail commit; ack}} C_{v+1}$$

CRAQ-style write-all, read-any

Definition (CRAQ-style replication)
Writes propagate through every replica in an ordered chain; reads may use any replica whose state is safe for the requested semantics.

$$C_v \xrightarrow{\text{prepare}} (C_v, U_{v+1}) \xrightarrow{\text{tail commit; ack}} C_{v+1}$$

- The head returns success to the client only after the returning acknowledgment reaches it — not when the first replica gets new bytes.

CRAQ-style write-all, read-any

Definition (CRAQ-style replication)
Writes propagate through every replica in an ordered chain; reads may use any replica whose state is safe for the requested semantics.

$$C_v \xrightarrow{\text{prepare}} (C_v, U_{v+1}) \xrightarrow{\text{tail commit; ack}} C_{v+1}$$

- The head returns success to the client only after the returning acknowledgment reaches it — not when the first replica gets new bytes.
- A target holding both C_v and U_{v+1} returns a special status rather than silently picking a version.

CRAQ-style write-all, read-any

Definition (CRAQ-style replication)
Writes propagate through every replica in an ordered chain; reads may use any replica whose state is safe for the requested semantics.

$$C_v \xrightarrow{\text{prepare}} (C_v, U_{v+1}) \xrightarrow{\text{tail commit; ack}} C_{v+1}$$

- The head returns success to the client only after the returning acknowledgment reaches it — not when the first replica gets new bytes.
- A target holding both C_v and U_{v+1} returns a special status rather than silently picking a version.

"A replica has the new bytes" is weaker than "the normal write path completed."

Five checkpoint publication states

Pending → Replicated → Committed → Published → Restored

Five checkpoint publication states

Pending $\rightarrow$ Replicated $\rightarrow$ Committed $\rightarrow$ Published $\rightarrow$ Restored

- **Committed** means an immutable manifest M_g has been validated — it is *not* yet the restore source.

Five checkpoint publication states

Pending $\rightarrow$ Replicated $\rightarrow$ Committed $\rightarrow$ Published $\rightarrow$ Restored

- **Committed** means an immutable manifest M_g has been validated — it is *not* yet the restore source.
- **Published** is the one atomic namespace operation that moves selector π from the old manifest to M_g.

Five checkpoint publication states

Pending $\rightarrow$ Replicated $\rightarrow$ Committed $\rightarrow$ Published $\rightarrow$ Restored

- **Committed** means an immutable manifest M_g has been validated — it is *not* yet the restore source.
- **Published** is the one atomic namespace operation that moves selector π from the old manifest to M_g.
- Unless g reaches Published, π never changes, even if g is abandoned.

Five checkpoint publication states

Pending $\rightarrow$ Replicated $\rightarrow$ Committed $\rightarrow$ Published $\rightarrow$ Restored

- **Committed** means an immutable manifest M_g has been validated — it is *not* yet the restore source.
- **Published** is the one atomic namespace operation that moves selector π from the old manifest to M_g.
- Unless g reaches Published, π never changes, even if g is abandoned.

This state machine is a chapter-defined deployment rule built on 3FS semantics — not a literal 3FS API. The chunk transitions underneath it are source-backed.

Target-local recovery and client I/O

From concept to code: writing intent before commit

A storage target must not confuse newly written bytes with the metadata that makes them live.

from `3FS/src/storage/chunk_engine/src/types/chunk_meta.rs`

```
pub struct ChunkMeta {
    pub pos: Position,
    pub chain_ver: u32,
    pub chunk_ver: u32,
    ...
    pub uncommitted: bool,
}
```

From concept to code: writing intent before commit

A storage target must not confuse newly written bytes with the metadata that makes them live.

from `3FS/src/storage/chunk_engine/src/types/chunk_meta.rs`

```
pub struct ChunkMeta {
    pub pos: Position,
    pub chain_ver: u32,
    pub chunk_ver: u32,
    ...
    pub uncommitted: bool,
}
```

`update_chunk` rejects a stale chain version, a committed/skipped update version, or a mismatched tag *before* publishing new metadata; only the RocksDB commit clears `uncommitted`. [INSPECTED: 3FS `src/storage/chunk_engine/src/core/engine.rs`]

FUSE and native USRBIO: one pathname, two data paths

Path	Metadata	Data I/O
FUSE client	kernel FUSE → 3FS FUSE process	ordinary read/write calls

FUSE and native USRBIO: one pathname, two data paths

Path	Metadata	Data I/O
FUSE client	kernel FUSE → 3FS FUSE process	ordinary read/write calls
Native client	same FUSE-facing `open`	registered fd, `Iov`/`Ior`, explicit CQEs

FUSE and native USRBIO: one pathname, two data paths

Path	Metadata	Data I/O
FUSE client	kernel FUSE → 3FS FUSE process	ordinary read/write calls
Native client	same FUSE-facing `open`	registered fd, `Iov`/`Ior`, explicit CQEs

- The native client runs *inside* the 3FS FUSE process — USRBIO is not a third peer client.

FUSE and native USRBIO: one pathname, two data paths

Path	Metadata	Data I/O
FUSE client	kernel FUSE → 3FS FUSE process	ordinary read/write calls
Native client	same FUSE-facing `open`	registered fd, `Iov`/`Ior`, explicit CQEs

- The native client runs *inside* the 3FS FUSE process — USRBIO is not a third peer client.
- Posting work is not completion: only a CQE from `hf3fs_wait_for_ios` proves an I/O finished.

FUSE and native USRBIO: one pathname, two data paths

Path	Metadata	Data I/O
FUSE client	kernel FUSE → 3FS FUSE process	ordinary read/write calls
Native client	same FUSE-facing `open`	registered fd, `Iov`/`Ior`, explicit CQEs

- The native client runs *inside* the 3FS FUSE process — USRBIO is not a third peer client.
- Posting work is not completion: only a CQE from `hf3fs_wait_for_ios` proves an I/O finished.

Metadata setup is shared; payload submission and completion are not — so the two paths are not interchangeable in a benchmark report.

Workloads, benchmarks, and operations

The checkpoint ledger, worked

Quantity	Toy value	Meaning
B_{ckpt}	$9.6\,\mathrm{TiB}$	logical checkpoint payload

The checkpoint ledger, worked

Quantity	Toy value	Meaning
B_{ckpt}	$9.6\,\mathrm{TiB}$	logical checkpoint payload
β_{write}	$4.1\,\mathrm{GiB/s}$	required write rate, $\rho = 1.5$

The checkpoint ledger, worked

Quantity	Toy value	Meaning
B_{ckpt}	$9.6\,\mathrm{TiB}$	logical checkpoint payload
β_{write}	$4.1\,\mathrm{GiB/s}$	required write rate, $\rho = 1.5$
β_{restore}	$24.6\,\mathrm{GiB/s}$	rate for a 10-minute restore

The checkpoint ledger, worked

Quantity	Toy value	Meaning
B_{ckpt}	$9.6\,\mathrm{TiB}$	logical checkpoint payload
β_{write}	$4.1\,\mathrm{GiB/s}$	required write rate, $\rho = 1.5$
β_{restore}	$24.6\,\mathrm{GiB/s}$	rate for a 10-minute restore
$rR\,B_{\mathrm{ckpt}}$	$230\,\mathrm{TiB}$	raw replicated payload, $r = 3$, $R = 8$

The checkpoint ledger, worked

Quantity	Toy value	Meaning
B_{ckpt}	$9.6\,\mathrm{TiB}$	logical checkpoint payload
β_{write}	$4.1\,\mathrm{GiB/s}$	required write rate, $\rho = 1.5$
β_{restore}	$24.6\,\mathrm{GiB/s}$	rate for a 10-minute restore
$rR\,B_{\mathrm{ckpt}}$	$230\,\mathrm{TiB}$	raw replicated payload, $r = 3$, $R = 8$

Fire-Flyer 2 reports a ∼5-minute checkpoint cadence and $> 10\,\mathrm{GiB/s}$ per node — source-reported context, not a reproduction of this toy ledger or a released-3FS-commit measurement. [REPORTED: Fire-Flyer AI-HPC paper, arXiv:2408.14158]

Match the benchmark to the workload, and know when not to deploy

Surface	Scope
README peak-read / GraySort / KV-cache	Source-reported; cluster- and setup-specific

Match the benchmark to the workload, and know when not to deploy

Surface	Scope
README peak-read / GraySort / KV-cache	Source-reported; cluster- and setup-specific
fio USRBIO / storage-bench	Native-client batching or component-level network/storage modes

Match the benchmark to the workload, and know when not to deploy

Surface	Scope
README peak-read / GraySort / KV-cache	Source-reported; cluster- and setup-specific
fio USRBIO / storage-bench	Native-client batching or component-level network/storage modes
DataStorage P model	Bounded schedules; replica-content assertions are *commented out* in the pinned monitor

Match the benchmark to the workload, and know when not to deploy

Surface	Scope
README peak-read / GraySort / KV-cache	Source-reported; cluster- and setup-specific
fio USRBIO / storage-bench	Native-client batching or component-level network/storage modes
DataStorage P model	Bounded schedules; replica-content assertions are *commented out* in the pinned monitor

Do not default to 3FS for simple scratch storage, unowned RDMA/FoundationDB operations, or a latency-critical cache with no measured request-mix trace.

Match the benchmark to the workload, and know when not to deploy

Surface	Scope
README peak-read / GraySort / KV-cache	Source-reported; cluster- and setup-specific
fio USRBIO / storage-bench	Native-client batching or component-level network/storage modes
DataStorage P model	Bounded schedules; replica-content assertions are *commented out* in the pinned monitor

Do not default to 3FS for simple scratch storage, unowned RDMA/FoundationDB operations, or a latency-critical cache with no measured request-mix trace.

This project has no 3FS deployment: peak-read, KV-cache, USRBIO, and checkpoint-restore reproduction are administratively closed, not measured zero or failed results. [OBSERVED: bounded repository-model and source-surface checks only]

Concept-to-code map for the chapter

- **lease fencing and routing snapshot** → `3FS/src/mgmtd/service/MgmtdState.cc`

Concept-to-code map for the chapter

- **lease fencing and routing snapshot** → `3FS/src/mgmtd/service/MgmtdState.cc`
- **disk–group placement and chain generation** → `3FS/deploy/data_placement/src/setup/gen_chain_table.py`

Concept-to-code map for the chapter

- **lease fencing and routing snapshot** → `3FS/src/mgmtd/service/MgmtdState.cc`
- **disk–group placement and chain generation** → `3FS/deploy/data_placement/src/setup/gen_chain_table.py`
- **local chunk commit and writing intent** → `3FS/src/storage/chunk_engine/src/core/engine.rs`

Concept-to-code map for the chapter

- **lease fencing and routing snapshot** → `3FS/src/mgmtd/service/MgmtdState.cc`
- **disk–group placement and chain generation** → `3FS/deploy/data_placement/src/setup/gen_chain_table.py`
- **local chunk commit and writing intent** → `3FS/src/storage/chunk_engine/src/core/engine.rs`
- **USRBIO buffers, rings, and completion** → `3FS/src/lib/api/UsrbIo.md`

Concept-to-code map for the chapter

- **lease fencing and routing snapshot** → `3FS/src/mgmtd/service/MgmtdState.cc`
- **disk–group placement and chain generation** → `3FS/deploy/data_placement/src/setup/gen_chain_table.py`
- **local chunk commit and writing intent** → `3FS/src/storage/chunk_engine/src/core/engine.rs`
- **USRBIO buffers, rings, and completion** → `3FS/src/lib/api/UsrbIo.md`
- **operation replay and reliable update** → `3FS/src/storage/service/ReliableUpdate.cc`

Concept-to-code map for the chapter

- **lease fencing and routing snapshot** → `3FS/src/mgmtd/service/MgmtdState.cc`
- **disk–group placement and chain generation** → `3FS/deploy/data_placement/src/setup/gen_chain_table.py`
- **local chunk commit and writing intent** → `3FS/src/storage/chunk_engine/src/core/engine.rs`
- **USRBIO buffers, rings, and completion** → `3FS/src/lib/api/UsrbIo.md`
- **operation replay and reliable update** → `3FS/src/storage/service/ReliableUpdate.cc`

Bytes land on disk long before a generation is safe to restore.

Next: smallpond builds dependency-aware dataframe tasks on top of this same storage protocol (Chapter 31).